

Introdução à Programação na Web — Resumo de Matéria

Disciplina: Introdução à Programação na Web / Programação na Internet

Instituto: ISEL — Instituto Superior de Engenharia de Lisboa

Cursos: Licenciatura em Engenharia Informática e de Computadores / Redes e Telecomunicações

Cobertura: Testes globais de 2021/2022 a 2024/2025 (Época Normal e Recurso)

Índice

1. [Protocolo HTTP](#)
2. [URIs e URLs](#)
3. [HTML — Estrutura de Documentos](#)
4. [CSS — Selectors](#)
5. [JavaScript — Fundamentos](#)
6. [Node.js — Ambiente de Execução e Event Loop](#)
7. [Programação Assíncrona — Callbacks, Promises e async/await](#)
8. [Fetch API](#)
9. [Express.js — Framework Web em Node.js](#)
10. [Handlebars — Motor de Templates](#)
11. [Autenticação e Segurança \(Headers HTTP\)](#)
12. [Arquitetura de Aplicações Web — Separação de Responsabilidades](#)

1. Protocolo HTTP

1.1 O que é o HTTP

O **HTTP (HyperText Transfer Protocol)** é o protocolo de comunicação da Web. Segue um modelo cliente-servidor baseado em pedidos (*requests*) e respostas (*responses*). Cada mensagem HTTP é constituída por uma linha inicial, cabeçalhos (*headers*) e, opcionalmente, um corpo (*body*).

1.2 Métodos HTTP

Cada pedido HTTP usa um **método** que indica a intenção da operação sobre o recurso identificado pelo URI.

Método	Semântica principal	Safe	Idempotente	Body típico
GET	Obter representação de recurso	✓	✓	Não
POST	Criar recurso (URI desconhecido) ou submeter dados	✗	✗	Sim
PUT	Criar ou substituir recurso (URI conhecido)	✗	✓	Sim
DELETE	Remover recurso	✗	✓	Não (normalmente)

Definições formais:

- **Safe (seguro):** um pedido é *safe* se não altera o estado da aplicação no servidor — é uma operação de **leitura apenas** (*read-only*). GET é o método safe por excelência.
- **Idempotente:** um pedido é *idempotente* se pode ser repetido múltiplas vezes produzindo sempre o mesmo resultado no estado do servidor. GET, PUT e DELETE são idempotentes; POST **não** é.

⚠ **Nota crítica:** Usar `GET /resource/123/delete` para remover um recurso é **incorreto** segundo a especificação HTTP. GET deve ser usado apenas para obter uma representação do recurso identificado pelo URI — neste caso, seria uma representação do recurso `/resource/123/delete`, não uma remoção.

1.3 Formas de Parametrizar um Pedido HTTP

Um pedido HTTP pode transportar dados relevantes para a aplicação através de:

- **URI (path)** — parâmetros embutidos no caminho, ex: `/users/123`
- **Query string** — parâmetros após `?`, ex: `/users?name=Ana`
- **Headers** — cabeçalhos do pedido, ex: `Authorization`, `Content-Type`
- **Body** — corpo do pedido (comum em POST e PUT)

⚠ "Middleware arguments" **não** é uma forma válida de parametrizar um pedido HTTP.

1.4 Status Codes

Os **status codes** das respostas HTTP são organizados em séries:

Série	Significado
2xx	Sucesso — a resposta corresponde ao pedido realizado
3xx	Redirecionamento — o cliente deve fazer algo adicional para obter o recurso
4xx	Erro do cliente — o pedido contém erro ou não pode ser satisfeito por culpa do cliente
5xx	Erro do servidor — o pedido não foi satisfeito por falha no servidor

Códigos mais relevantes e quando usá-los:

- **200 OK** — pedido processado com sucesso; recurso retornado.
- **201 Created** — recurso criado com sucesso (tipicamente resposta a POST ou PUT).
- **204 No Content** — processamento com sucesso, sem corpo na resposta.
- **303 See Other** — redirecionamento: o browser deve fazer um GET para o URL indicado no header `Location`. **Usado no padrão Post/Redirect/Get.**
- **400 Bad Request** — o pedido não pode ser processado **por erro do cliente** (ex: parâmetro obrigatório em falta, dados inválidos).
- **401 Unauthorized** — o cliente **não apresentou credenciais válidas** (não autenticado).
- **403 Forbidden** — o cliente está autenticado mas **não tem permissão** para aceder ao recurso.
- **404 Not Found** — o recurso identificado pelo URI **não existe**.
- **500 Internal Server Error** — erro interno do servidor (ex: base de dados inacessível, serviço externo em baixo).

- **502 Bad Gateway** — serviço externo/dependência do servidor está indisponível.

Regra de ouro para distinguir 4xx de 5xx:

- Se o problema é do **cliente** (pedido mal formado, credenciais erradas, recurso não existe) → **4xx**
- Se o problema é do **servidor** (crash, base de dados em baixo, serviço externo falhado) → **5xx**

1.5 Headers HTTP Importantes

Header	Direção	Função
Content-Type	Pedidos e respostas	Especifica o tipo de conteúdo do corpo da mensagem (ex: <code>application/json</code> , <code>text/html</code> , <code>application/csv</code>)
Authorization	Só pedidos	Fornece credenciais de autenticação (ex: token Bearer do utilizador)
Location	Só respostas	URL de destino para redirecionamentos (usado com 3xx)
Cookie	Só pedidos	Envia cookies guardados no browser para o servidor
Set-Cookie	Só respostas	Instrui o browser a guardar um cookie no cliente

Content-Type: `application/json` deve ser enviado em pedidos POST/PUT com body em JSON. É **obrigatório** para que o servidor saiba interpretar o corpo.

1.6 Padrão Post/Redirect/Get (PRG)

Quando um formulário HTML é submetido por POST e o servidor responde diretamente com uma página HTML (sem redirecionar), o browser apresenta um aviso ao utilizador ao fazer *refresh* — porque iria reenviar o POST.

A solução é o padrão **Post/Redirect/Get**:

1. Cliente submete um POST
2. Servidor processa e responde com **303 See Other** + header `Location`
3. Browser faz automaticamente um **GET** para o URL indicado
4. Servidor responde com a página — agora um *refresh* apenas repete o GET (seguro)

1.7 Cookies HTTP

Os **cookies** são um mecanismo pelo qual o **servidor guarda dados no cliente** (browser) e os recupera em pedidos futuros. O servidor envia `Set-Cookie` na resposta; o browser devolve `Cookie` nos pedidos seguintes. São usados para manter sessão, preferências do utilizador, etc. **Não** servem para guardar informação sensível diretamente, nem para autenticar por si sós.

2. URIs e URLs

2.1 Estrutura de um URI

Um **URI (Uniform Resource Identifier)** pode ser constituído pelos seguintes componentes:

```
scheme://host:port/path?query#fragment
```

- **scheme** — protocolo (ex: `http`, `https`)
- **host** — nome do servidor (ex: `www.example.com`)
- **port** — porta (opcional; ex: `3000`, `9000`)
- **path** — caminho para o recurso (ex: `/products/123`)
- **query string** — parâmetros adicionais após `?` (ex: `id=123&category=electronics`)
- **fragment** — âncora no documento, após `#` (ex: `#biography`) — **não é enviado ao servidor**

Exemplos de análise:

- `http://www.example.com/products?id=123&category=electronics`
 - **path:** `/products`
 - **query string:** `id=123&category=electronics` (dois parâmetros: `id` e `category`)
 - `www.example.com` identifica o **servidor**, não um recurso
- `http://some.host/groups/1?prev=/movies/2`
 - **path:** `/groups/1`
 - **query:** `prev=/movies/2`
 - O fragmento `/movies/2` está dentro do **valor** do parâmetro `prev`

⚠ "middleware arguments" não faz parte da estrutura de um URI/URL.

2.2 Rotas em Express e URI

No Express, as **rotas** são identificadas pela combinação de **path do URI + método HTTP**. A query string e o fragment não fazem parte da identificação da rota.

3. HTML — Estrutura de Documentos

3.1 Estrutura Básica

Um documento HTML é composto por **elementos e texto**. Os elementos têm:

- **Marca de início** (`<tag>`) e, opcionalmente, **marca de fim** (`</tag>`)
- **Atributos** na marca de início — têm **nome** e podem ter **valor**

```
<html>           <!-- elemento raiz; único elemento raiz do documento -->
  <head>          <!-- metadados: título, links para CSS, scripts -->
    <title>Página</title>
  </head>
  <body>          <!-- conteúdo visível -->
    <p class="info" id="intro">Olá</p>
  </body>
</html>
```

3.2 Atributos `id` e `class`

- **id** — deve ter um **valor único** num documento; identifica um único elemento.
- **class** — pode ser **partilhado por múltiplos elementos**; agrupa elementos para estilo ou comportamento.

3.3 Elemento `form`

O elemento `<form>` permite ao utilizador submeter dados ao servidor. Atributos principais:

- **method** — método HTTP usado na submissão (`get` ou `post`)
- **action** — URI para onde o pedido é enviado (por omissão, o mesmo URI da página)

Para submissão automática via botão:

```
<form method="post" action="/endpoint">
  <input type="text" name="campo">
  <input type="submit"> <!-- ou <button type="submit"> -->
</form>
```

`<input type="submit">` é o elemento correto para um botão de submissão. O atributo `name` nos campos é o que define o nome do parâmetro enviado no body do pedido.

4. CSS — Selectors

Os **seletores CSS** definem a que elementos se aplica uma regra de estilo. Os mais relevantes:

Seletor	Significado	Exemplo
<code>a</code>	Seleciona todos os elementos de tipo <code>a</code>	<code>div</code> , <code>button</code> , <code>input</code>
<code>.danger</code>	Seleciona todos os elementos com a classe <code>danger</code>	<code>.column-xs</code>
<code>#funny</code>	Seleciona o elemento com o id <code>funny</code>	<code>#user1</code>
<code>div</code> <code>.danger</code>	Seleciona todos os elementos com classe <code>danger</code> que são descendentes de <code>div</code>	

Regra de ouro:

- `.classe` → começa com **ponto**
- `#id` → começa com **cardinal**
- `elemento` → nome do elemento sem prefixo

```
/* Seleciona todos os elementos com classe "note" */
.note { color: red; }
```

```
/* Seleciona o elemento com id "header" */
#header { font-size: 24px; }
```

Em JavaScript no browser:

- `document.querySelector(".note")` → primeiro elemento com classe `note`
- `document.querySelectorAll(".note")` → **todos** os elementos com classe `note`
- `document.querySelector("#user1")` → elemento com id `user1`
- `document.querySelectorAll("button")` → **todos** os elementos `<button>`
- `document.querySelectorAll("div.special")` → todos os `<div>` com classe `special`

5. JavaScript — Fundamentos

5.1 Funções como Valores (Higher-Order Functions)

Em JavaScript, as funções são **cidadãos de primeira classe** — podem ser passadas como argumentos, retornadas por outras funções e atribuídas a variáveis. Uma função que recebe ou retorna outra função chama-se **função de ordem superior**.

- **Callback:** uma função passada como argumento a outra função, para ser chamada mais tarde.
- **Arrow function:** sempre **anónima** `((x) => x + 1)`; não tem o seu próprio `this`.

5.2 Closures

Um **closure** é um mecanismo que **encapsula o ambiente léxico** de uma função, permitindo que a função acesse as variáveis do seu contexto de definição mesmo depois de essa função ter retornado.

Um closure forma-se **sempre que uma função é definida dentro de outra função e é acessível do exterior** — nesse momento a função interna "captura" as variáveis da função exterior.

```
function contador() {
  let n = 0;
  return function() { return ++n; }; // captura `n` via closure
}
const c = contador();
c(); // 1
c(); // 2
```

5.3 O `this` em JavaScript

O valor de `this` dentro de uma função depende de **como a função é chamada**, não de onde é definida:

Como a função é chamada	Valor de <code>this</code>
Como função global	<code>undefined</code> (strict mode) ou objeto global
Como método de um objeto	O objeto usado na chamada

Como a função é chamada	Valor de <code>this</code>
Como construtor (<code>new</code>)	O novo objeto criado
Via <code>.call()</code> ou <code>.apply()</code>	O objeto arbitrário passado como argumento

⚠ **Armadilha clássica:** se um método de um objeto é atribuído a outra variável e chamado sem o objeto, perde o contexto. Ex: `this.removeItem = items.pop` — ao chamar `coll.removeItem()`, o `this` dentro de `pop` não será `items`, mas sim `coll`, logo o comportamento é inesperado. A solução é usar uma função envolvente: `this.removeItem = () => items.pop()`.

5.4 Métodos de Array

Método	Retorna	Recebe
<code>map(fn)</code>	Novo array com mesmo número de elementos, transformados	Função de transformação
<code>filter(fn)</code>	Novo array com menor ou igual número de elementos	Função predicado (<code>Boolean</code>)
<code>forEach(fn)</code>	<code>undefined</code>	Função a executar por cada elemento
<code>find(fn)</code>	Um elemento do array ou <code>undefined</code>	Função predicado

5.5 Protótipos

JavaScript usa **herança baseada em protótipos**. Cada objeto tem uma referência para um protótipo, e a pesquisa de propriedades sobe a cadeia de protótipos até encontrar a propriedade ou chegar a `null`.

5.6 Funções Construtoras

Uma função chamada com `new` comporta-se como construtora: cria um novo objeto, define `this` para esse objeto, e retorna-o implicitamente.

```
function Ponto(x, y) {
  this.x = x;
  this.y = y;
}
const p = new Ponto(1, 2); // { x: 1, y: 2 }
```

6. Node.js — Ambiente de Execução e Event Loop

6.1 O que é o Node.js

Node.js é um **ambiente de execução (runtime) JavaScript** fora do browser, baseado no motor V8 do Chrome. Permite executar JavaScript no servidor. **Não** é uma linguagem, uma biblioteca, nem uma base de dados.

6.2 Event Loop e Modelo de Concorrência

Node.js é **single-threaded** e usa um modelo de I/O **não-bloqueante** baseado no **event loop**:

- O código JavaScript corre num único thread.
- Operações de I/O (leitura de ficheiros, pedidos de rede) são delegadas ao sistema operativo — quando terminam, um **callback** é colocado na fila de eventos para ser executado.
- O event loop só processa o próximo evento quando o código JavaScript em execução terminar.

Consequência crítica: um **while** infinito (ou de longa duração) **bloqueia o event loop** — nenhum callback, timer ou pedido HTTP será processado enquanto o ciclo estiver a correr. Os **setTimeout** registados antes do **while** nunca executarão.

```
let v = 1;
setTimeout(() => { v = 0 }, 3000); // este callback NUNCA corre
while (v) { /* bloqueia o event loop para sempre */ }
console.log("Nunca chega aqui");
```

Node.js pode ter funções síncronas — apenas as operações de I/O têm de ser assíncronas para não bloquearem o event loop. Funções puramente computacionais podem ser síncronas.

6.3 setTimeout

setTimeout(callback, ms) agenda a execução do **callback** após, **no mínimo**, **ms** milissegundos. O callback só corre quando o event loop estiver livre.

6.4 package.json e npm

O ficheiro **package.json** é o manifesto de uma aplicação Node.js. Serve para:

- **Registar as dependências** da aplicação (**dependencies**, **devDependencies**)
- **Distinguir dependências de execução de dependências de desenvolvimento**
- **Definir comandos de execução** (**scripts**)

⚠ **Não** serve para armazenar objetos JSON recebidos em pedidos HTTP.

O **npm (Node Package Manager)** simplifica a instalação, partilha e gestão de módulos de terceiros (*third-party*). Não é um sistema de controlo de versões.

6.5 Módulos

Node.js suporta dois sistemas de módulos:

- **CommonJS:** `const x = require('./modulo') / module.exports = ...`
- **ES Modules:** `import x from './modulo.mjs' / export default ...`

7. Programação Assíncrona — Callbacks, Promises e async/await

7.1 Callbacks

A forma mais simples de lidar com operações assíncronas: passa-se uma função (callback) que será chamada quando a operação terminar. Problema: encadeamento de callbacks cria o *callback hell* — código difícil de ler e manter.

7.2 Promises

Uma **Promise** representa um valor que pode estar disponível agora, no futuro, ou nunca. Tem três estados: *pending*, *fulfilled* (resolvida com sucesso) e *rejected* (resolvida com erro).

Métodos principais:

Método	O que faz
<code>then(fn)</code>	Executa <code>fn</code> quando a Promise é resolvida com sucesso ; retorna uma nova Promise
<code>catch(fn)</code>	Executa <code>fn</code> quando a Promise é rejeitada
<code>finally(fn)</code>	Executa <code>fn</code> independentemente do resultado (sucesso ou erro)
<code>Promise.resolve(val)</code>	Retorna uma Promise já resolvida com <code>val</code>
<code>Promise.reject(err)</code>	Retorna uma Promise já rejeitada com <code>err</code>
<code>Promise.all(arr)</code>	Retorna uma Promise que resolve com um array de valores quando todas as Promises do array resolverem; rejeita logo que uma rejeite
<code>Promise.allSettled(arr)</code>	Retorna uma Promise que resolve quando todas terminarem, independentemente de sucesso ou erro; cada posição contém <code>{status, value}</code>

`then()` sempre retorna uma Promise — nunca retorna `undefined`, `null` ou `String`.

Encadeamento:

```
fetch(url)
  .then(response => response.json()) // retorna Promise
  .then(data => console.log(data))   // recebe o objeto
  .catch(err => console.error(err));
```

Execução em paralelo com `Promise.all`:

```
const [r1, r2] = await Promise.all([
  operacao1(), // corre em paralelo
  operacao2()  // corre em paralelo
]);
// r1 e r2 disponíveis aqui
```

7.3 async/await

`async` e `await` são **açúcar sintático** sobre Promises — tornam o código assíncrono mais legível, sem alterar o comportamento.

Regras fundamentais:

- `async` antes de uma função: a função **retorna sempre uma Promise**, mesmo que o `return` seja um valor simples.
- `await` deve ser usado **antes de uma expressão que produz uma Promise**, e só pode ser usado **dentro de uma função `async`**.
- Uma função tem de ser `async` **obrigatoriamente** se usar `await` no seu corpo.
- Uma função que retorna uma Promise **não precisa** de ser `async` (pode usar `.then()` explicitamente).

```
// Com async/await
async function f1(p1, p2) {
  const v1 = await p1;
  const v2 = await p2;
  return v1 + v2;
}

// Equivalente com métodos de Promise
function f1(p1, p2) {
  return p1.then(v1 => p2.then(v2 => v1 + v2));
}
```

Uma função `async` é executada de forma síncrona até encontrar o primeiro `await`, altura em que suspende e devolve o controlo ao chamador. Portanto, `console.log(f3())` onde `f3` é `async` imprime uma **Promise**, não o valor.

`await` sequencial vs. paralelo:

```
// Sequencial – cada await espera que o anterior termine
async function sequencial() {
  const r1 = await op1(); // espera op1
  const r2 = await op2(); // só começa depois de op1
}

// Paralelo – ambas as operações começam em simultâneo
async function paralelo() {
  const [r1, r2] = await Promise.all([op1(), op2()]);
}
```

8. Fetch API

A **Fetch API** fornece uma interface para **enviar pedidos HTTP e receber respostas de forma assíncrona**, tanto no browser como em Node.js (com `node-fetch` ou nativamente).

8.1 Funcionamento

```
fetch(uri, options)
  .then(response => response.json()) // response.json() retorna uma Promise
  .then(data => console.log(data));
```

Elemento	O que é
<code>fetch(uri, options)</code>	Retorna uma Promise resolvida com um objeto Response
<code>options</code>	Objeto com todos os dados do pedido: <code>method</code> , <code>headers</code> , <code>body</code> , etc.
Objeto <code>response</code>	Contém todos os dados da resposta (status, headers, body)
<code>response.json()</code>	Retorna uma Promise que resolve com um objeto JS (se o body for JSON válido)

8.2 Exemplo de pedido POST equivalente à submissão de um formulário HTML

```
fetch('/endpoint', {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: 'campo1=valor1&campo2=valor2'
});
```

Para enviar JSON:

```
fetch('/api/resource', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nome: 'Ana', idade: 25 })
});
```

9. Express.js — Framework Web em Node.js

9.1 Conceitos Fundamentais

Express é uma framework minimalista para construir aplicações web e APIs em Node.js. Uma aplicação Express é um conjunto de **middlewares e handlers** registrados para responder a pedidos HTTP.

```
import express from 'express';
const app = express();
app.listen(3000, () => console.log('Listening...'));
```

9.2 Rotas

Uma **rota** no Express é identificada pelo **path do URI** combinado com o **método HTTP**. Não inclui a query string.

```
app.get('/users/:id', handler);    // GET /users/123
app.post('/users', handler);       // POST /users
app.put('/users/:id', handler);    // PUT /users/123
app.delete('/users/:id', handler); // DELETE /users/123
```

Parâmetros de rota (`:param`) são partes variáveis do path — acessíveis em `req.params`.

⚠ A ordem de registo das rotas **importa**. O Express testa as rotas pela ordem em que foram registadas e usa a primeira que corresponder. `/tasks/top` deve ser registada **antes** de `/tasks/:id`, caso contrário `top` seria interpretado como um id.

9.3 Objeto Request (`req`)

Propriedade / Método	O que contém
<code>req.params</code>	Objeto com as componentes variáveis do path do URI (ex: <code>req.params.id</code>)
<code>req.query</code>	Objeto com os dados da query string (ex: <code>req.query.page</code>)
<code>req.body</code>	Objeto com o conteúdo do corpo do pedido (após parsing por middleware)
<code>req.get(header)</code>	Retorna o valor de um header do pedido (ex: <code>req.get('Content-Type')</code>)
<code>req.method</code>	Método HTTP do pedido (ex: <code>'GET'</code> , <code>'POST'</code>)
<code>req.path</code>	Path do URI do pedido

9.4 Objeto Response (`res`)

Método	O que faz
<code>res.send(data)</code>	Envia uma resposta (texto, HTML, buffer)
<code>res.json(obj)</code>	Envia <code>obj</code> serializado como JSON, com <code>Content-Type: application/json</code>
<code>res.status(code)</code>	Define o status code (encadeável: <code>res.status(400).json(...)</code>)
<code>res.redirect(url)</code>	Responde com 302 (ou 303) e <code>Location: url</code>
<code>res.render(view, data)</code>	Renderiza um template (ex: Handlebars) com os dados <code>data</code>
<code>res.end()</code>	Termina a resposta sem corpo

9.5 Middlewares

Um **middleware** é uma função com a assinatura (`req, res, next`) que:

- **Tem acesso** aos objetos `req` e `res`
- Pode **modificar** `req` e `res` (incluindo adicionar propriedades)
- Pode **interceptar** um pedido antes de chegar ao handler
- Deve chamar `next()` para passar o controlo ao próximo middleware/handler, ou terminar a resposta

```
function meuMiddleware(req, res, next) {  
  // faz algo com req ou res  
  next(); // passa para o próximo  
}  
app.use(meuMiddleware); // aplica a todos os pedidos  
app.use('/api', meuMiddleware); // aplica apenas a paths que começam com /api
```

`app.use(fn)` — regista `fn` como middleware (associa ao caminho especificado, ou a todos).

9.6 Middlewares Built-in do Express

- `express.json()` — faz parse do body dos pedidos com `Content-Type: application/json`, colocando o resultado em `req.body`. `app.use(express.json())` regista o **retorno** da função como middleware (não a função em si).
- `express.urlencoded({extended: ...})` — faz parse do body de formulários HTML (com `Content-Type: application/x-www-form-urlencoded`), colocando os dados em `req.body`.

⚠ Sem `express.json()`, `req.body` é `undefined` para pedidos com body em JSON. A causa mais comum de `TypeError: Cannot read properties of undefined (reading 'campo')`.

9.7 Padrão Post/Redirect/Get no Express

```
app.post('/degrees/:deg/course', (req, res) => {  
  degrees[req.params.deg].push(req.body.course);  
  // ❌ res.render('courses', {...}) — causa aviso de resubmissão no refresh  
  // ✅ Correto:  
  res.redirect('/degrees/' + req.params.deg);  
});
```

9.8 Tratamento de Erros em Promises

Promises rejeitadas **não tratadas** em handlers Express terminam o processo em versões antigas. Deve-se encadear `.catch()` ou usar `try/catch` com `async/await`:

```
app.get('/file', (req, res) => {  
  fs.readFile(req.path)  
    .then(data => res.send(data))  
    .catch(err => res.status(500).send('Erro ao ler ficheiro'));  
});
```

9.9 Construção de Middlewares Reutilizáveis (Padrão Constructor)

É comum criar uma **função construtora de middleware** que recebe configuração e retorna a função middleware:

```
// Construtor
function loggerConstructor(headers = []) {
  // retorna o middleware
  return function(req, res, next) {
    console.log(`${req.method} - ${req.path}`);
    next();
  };
}

// Uso
const logger = loggerConstructor(['Content-Type', 'User-Agent']);
app.use(logger);
```

Este padrão permite **injeção de dependências** (ex: passar um objeto de serviço ao middleware) e **reutilização** com diferentes configurações.

9.10 Módulos ES no Express

Com ES Modules (`.mjs`), usa-se `import/export` em vez de `require/module.exports`:

```
// service.mjs
const data = {};
export function getData() { return data; }
export default { getData };

// server.mjs
import express from 'express';
import service from './service.mjs';
```

10. Handlebars — Motor de Templates

O **Handlebars** é um motor de templates usado com Express para gerar HTML dinâmico no servidor.

10.1 Conceito

- **Entradas:** um **template** (ficheiro `.hbs`) + um **objeto** com dados
- **Saída:** **texto** (HTML renderizado)
- O template contém **texto estático e expressões Handlebars** entre `{{ }}`
- As expressões acedem às **propriedades do objeto** passado

```
<ul>
  {{#each courses}}
    <li>{{@index}} - {{this}}</li>
  {{/each}}
</ul>
```

10.2 Expressões e Blocos Principais

Expressão	Função
{{propriedade}}	Substitui pelo valor da propriedade
{{#each lista}}...{{/each}}	Itera sobre uma lista
@index	Índice atual dentro de um bloco {{#each}} — só pode ser usado dentro de #each
{{#unless condição}}... {{/unless}}	Renderiza o bloco condicionalmente (quando a condição é <i>falsy</i>)
{{#with objeto}}...{{/with}}	Altera o contexto do bloco para o objeto especificado

11. Autenticação e Segurança (Headers HTTP)

11.1 Header **Authorization**

O header **Authorization** é usado para **fornecer credenciais de autenticação** ao servidor. No contexto dos projetos práticos desta disciplina, é usado para **enviar o token** associado ao utilizador (tipicamente um token Bearer).

```
Authorization: Bearer <token>
```

11.2 Status Codes de Autenticação/Autorização

- **401 Unauthorized** — o cliente **não apresentou credenciais** ou as credenciais são **inválidas** (ex: login com password errada, token ausente ou expirado).
- **403 Forbidden** — o cliente está **autenticado** (credenciais válidas) mas **não tem permissão** para aceder ao recurso específico.

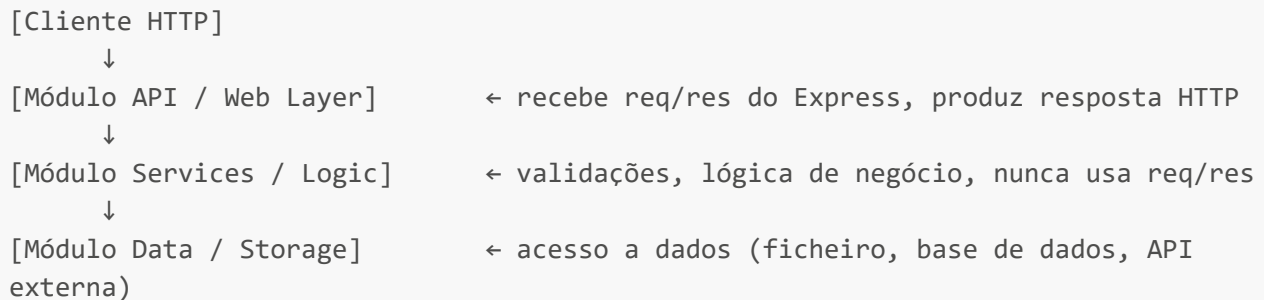
11.3 Passport.js [VERIFICAR]

O **Passport.js** é um middleware Node.js que **auxilia na autenticação** de aplicações. Integra-se com Express via **app.use()**.

12. Arquitetura de Aplicações Web — Separação de Responsabilidades

12.1 Estrutura em Camadas

As aplicações práticas desta disciplina seguem uma **arquitetura em camadas**:



12.2 Responsabilidades de Cada Camada

Módulo	Responsabilidade
*-api (Web Layer)	Obtém os dados do pedido (req), invoca serviços, produz resposta HTTP (res)
*-services	Realiza todas as validações dos dados recebidos; lógica de negócio; nunca recebe objetos req/res
*-data	Acesso a dados persistidos; pode ser base de dados, ficheiro, ou API externa

O módulo de serviços **nunca** deve receber nem manipular diretamente os objetos **Request** e **Response** do Express — isso é responsabilidade da camada API.

12.3 Injeção de Dependências em Módulos

O padrão typical é exportar uma **função construtora** que recebe as dependências como argumentos, retornando o objeto/módulo funcional. Isto desacopla as camadas e facilita testes.

```
// borga-services.mjs
export default function(data) {
  async function removeGroup(userId, groupId) {
    return data.removeGroup(userId, groupId);
  }
  return { removeGroup };
}
```

Termos-Chave

- **HTTP** — HyperText Transfer Protocol
- **URI / URL** — Uniform Resource Identifier / Locator
- **Query string** — parâmetros após **?** num URL
- **Fragment** — âncora após **#** num URL, não enviado ao servidor

- **Status code** — código numérico de 3 dígitos na resposta HTTP
- **Safe** — método HTTP que não altera o estado do servidor (ex: GET)
- **Idempotente** — método HTTP cujo efeito é o mesmo independentemente do número de repetições (ex: GET, PUT, DELETE)
- **Content-Type** — header que especifica o tipo de conteúdo do corpo da mensagem
- **Authorization** — header que transporta credenciais de autenticação
- **Location** — header que indica URL de redirecionamento
- **Cookie / Set-Cookie** — mecanismo para guardar dados do servidor no cliente
- **Post/Redirect/Get (PRG)** — padrão que evita resubmissão de formulários ao fazer refresh
- **Node.js** — ambiente de execução JavaScript no servidor
- **Event loop** — mecanismo de concorrência single-threaded do Node.js
- **npm** — Node Package Manager
- **package.json** — manifesto de uma aplicação Node.js
- **Express.js** — framework web minimalista para Node.js
- **Middleware** — função (`req`, `res`, `next`) que interceta pedidos HTTP
- **Handler** — função que processa um pedido e produz a resposta final
- **Route** — par (path, método HTTP) que identifica um handler no Express
- `req.params` — parâmetros variáveis do path da rota
- `req.query` — parâmetros da query string
- `req.body` — corpo do pedido (após parsing por middleware)
- `express.json()` — middleware que faz parse de body em JSON
- `express.urlencoded()` — middleware que faz parse de body de formulários HTML
- `res.redirect()` — responde com redirecionamento
- `res.render()` — renderiza um template Handlebars
- **Callback** — função passada como argumento para ser chamada mais tarde
- **Promise** — objeto que representa um valor assíncrono futuro
- `Promise.all()` — aguarda que todas as promises resolvam (execução em paralelo)
- `Promise.allSettled()` — aguarda todas as promises, mesmo as rejeitadas
- **async** — palavra reservada que faz uma função retornar sempre uma Promise
- **await** — suspende a execução até uma Promise resolver; só válido dentro de **async**
- **Closure** — função que captura o ambiente léxico onde foi definida
- **this** — contexto de execução de uma função, dependente do modo de chamada
- **Higher-order function** — função que recebe ou retorna outra função
- **Fetch API** — interface para pedidos HTTP assíncronos (browser e Node.js)
- **Handlebars** — motor de templates para geração de HTML no servidor
- `#each` — bloco Handlebars para iteração sobre listas
- `@index` — índice atual dentro de um bloco `#each` do Handlebars
- **CSS selector** — padrão para selecionar elementos HTML (`.classe`, `#id`, `elemento`)
- **id** — atributo HTML único por documento
- **class** — atributo HTML partilhável por múltiplos elementos
- **Separação de responsabilidades** — arquitetura em camadas: API, Services, Data
- **Passport.js** — middleware de autenticação para Express
- **ES Modules** — sistema de módulos moderno JavaScript (`import/export`)
- **CommonJS** — sistema de módulos clássico do Node.js (`require/module.exports`)